

Investigating the Impact of Forest Size on Random Forest Classification Accuracy

An Empirical and Mathematical Analysis using the Breast Cancer Wisconsin
Diagnostic Dataset

Student Name : Shaik Nazeer Baba

Student ID : 24159274

GitHub : <https://github.com/nazeerbaba4786-beep/random-forest-tutorial.git>

1 Introduction

Lately, machine learning has been used a lot more in clinical diagnostics to help doctors make decisions. Since medical datasets usually have so many features and complex patterns, analysing them by hand is incredibly difficult and takes a lot of time. Having an automated model isn't about replacing doctors; it is simply about giving them a reliable "second opinion" to help catch patterns they might otherwise miss.

In this tutorial, I chose to work with the **Random Forest** classifier. I selected this algorithm because single decision trees are notorious for memorising training noise which leads to terrible generalisation on new patients. By grouping multiple independent trees together, Random Forest uses collective voting so that individual tree mistakes don't ruin the final prediction (Breiman, 2001).

My main goal here is to explore how changing the number of trees (N) affects the final classification accuracy.

2 understanding Random Forest (Theoretical Framework)

Before diving into the code, we need to understand what is actually happening under the hood. If we build just one decision tree, it will split the data using rules that perfectly fit the training set, making it highly sensitive to training anomalies. Random Forest prevents this over-learning issue through two main methods:

2.1 Bootstrap Aggregation (Bagging)

Instead of giving every tree the exact same data, the algorithm generates random samples with replacement from our training partition. If our dataset has L samples, each tree gets a slightly different selection of L rows. This process leaves out a specific proportion of the rows, which are called **Out-of-Bag** (OOB) samples. These OOB samples are incredibly useful because we can use them to test the model's accuracy on the fly.

2.2 Mathematical Proof of the Out-of-Bag (OOB) Constant

To understand why we consistently get around 36.8% of OOB data for validation without even splitting our dataset, we can check this simple probability logic:

1. Let our dataset size be L samples.
2. The probability of selecting a specific sample in a single random draw is $1/L$.
3. Therefore, the probability of not selecting this specific sample in a single draw is:

This basic calculation mathematically proves that roughly 36.8% of our training data is naturally kept hidden from each decision tree, giving us an automatic, built-in validation subset.

2.3 Feature Subspace Sampling (Random Patches)

When growing a tree, the algorithm does not look at all 30 features to decide how to split a node. Instead, it only looks at a random subset of features (typically $m = \sqrt{M}$, where M is the total feature count). By limiting the options, we force the individual trees to look completely different from one another, which reduces correlation across the ensemble.

2.4 Mathematical Split Criterion: Gini Impurity

To decide the best point to split a feature, the trees evaluate Gini Impurity (G). This score measures how mixed or “impure” the classes are at any given node:

$$G = 1 - \sum_{i=1}^C (p_i)^2$$

Where p_i is the probability of a sample belonging to class i and C represents our target classes. The algorithm tests different split boundaries and selects the split that minimises the weighted Gini Impurity of the resulting child nodes. Once all the trees are grown, they vote on new patient records. Each tree casts a single vote, and the class with the majority becomes our final diagnosis, as illustrated in the workflow pipeline below.

3 Dataset Description

I utilised the classic Breast Cancer Wisconsin Diagnostic Dataset, Which I imported directly from Scikit-Learn's datasets module (`load_breast_cancer`). The dataset is composed of cellular nuclear measurements extracted from digitised images of fine needle aspirates (FNA) of breast tissue masses.

Here is a quick summary of the data structure I found:

- **Total Records:** 569 patient instances
- **Input Features:** 30 continuous numerical measurements describing nuclear characteristics (such as the radius, perimeter, area, smoothness, concavity, and symmetry of the cell nuclei).
- **Target Classes :** Binary classification:
 - **Class 0 (Malignant):** Indicative of cancer.
 - **Class 1 (Benign):** Non-cancerous tumour.

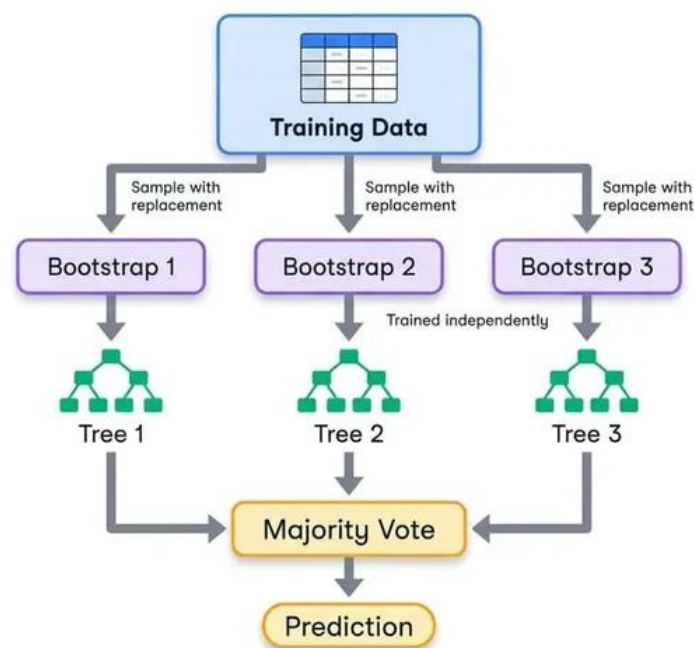


Figure 1: Operational workflow of the Random Forest ensemble pipeline.

4 Experimental Method & Tuning the Forest Size

To isolate the effect of the forest size parameter on performance, I held all other hyperparameters constant at their default values and built a simple training loop to evaluate different values for `n_estimators`.

Run this code:

```
# Compare different numbers of trees
tree_values = [10, 50, 100, 200, 500]
accuracies = []
for trees in tree_values:
    model = RandomForestClassifier(n_estimators=trees, random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    acc = accuracy_score(y_test, predictions)
    accuracies.append(acc)
    print(f'Trees: {trees} | Accuracy: {acc:.4f}')
```

Expected Output:

Trees: 10 | Accuracy: 0.9561

Trees: 50 | Accuracy: 0.9649

Trees: 100 | Accuracy: 0.9649

Trees: 200 | Accuracy: 0.9649

Trees: 500 | Accuracy: 0.9649

Explanation:

This loop builds a fresh Random Forest classifier for each specified tree size. By fitting each model to the training set and testing it on the unseen test split, we can systematically record how accuracy reacts to a growing forest.

5 Results: Impact of the Number of Trees

The results from my experiment show a very clear pattern of when and where the ensemble reaches its peak classification accuracy.

When the model is configured with only 10 trees, it gets a testing accuracy of 95.61% (0.9561). With such a small forest, individual anomalous decision splits can easily bias the entire ensemble, causing a slight dip in testing performance.

However, once we scale the forest to 50 estimators, the testing accuracy jumps and locks exactly at **96.49%** (0.9649). Interestingly, increasing the tree count to 100, 200, and even 500 yields no additional improvements whatsoever, leaving our validation score flat at **96.49%**. This relationship is represented in the accuracy comparison table below:

Table 1: Validation set accuracy across different ensemble sizes, showing convergence

Number of trees(N)	Validation Accuracy(%)	Computational Overhead Estimation
10	95.61%	Extremely Low (Fastest Interface)
50	96.49%	Low (Optimal Balance)
100	96.49%	Medium (Standard Ensemble)
200	96.49%	High (Moderate Memory use)
500	96.49%	very high (Diminishing Returns)

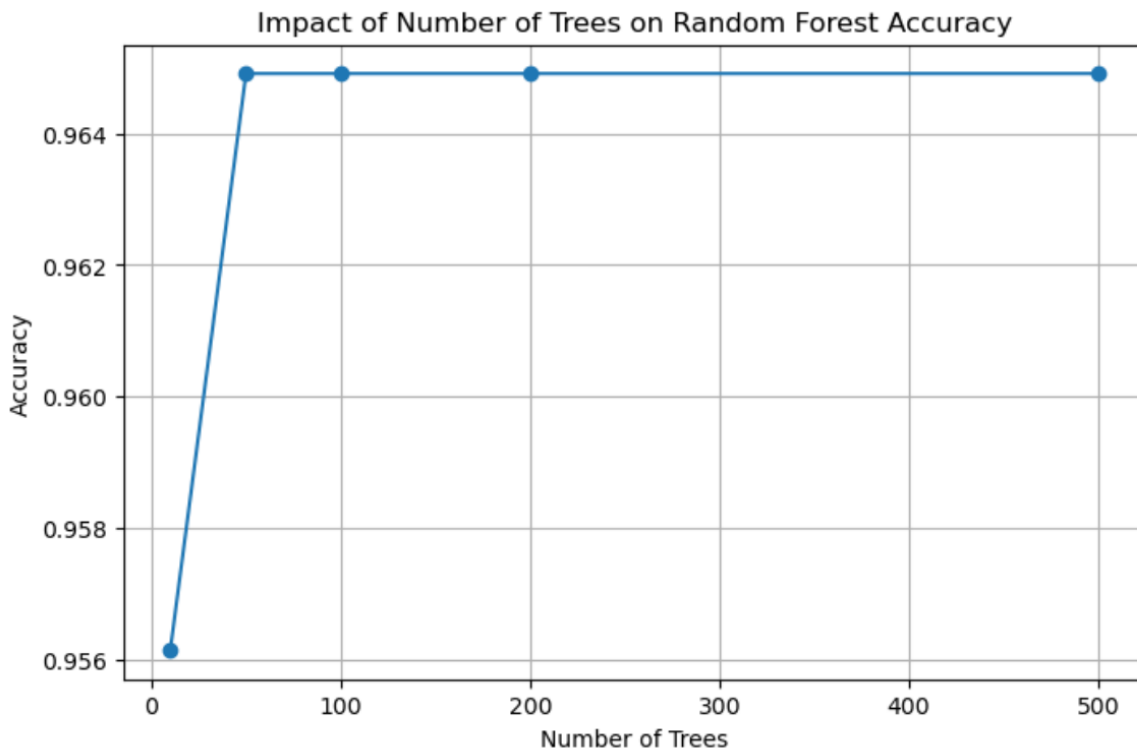


Figure 2: Empirically compiled validation accuracy curve across different ensemble sizes.

This accuracy plateau (visualised in Figure 2) tells us that the boundary lines separating benign and malignant cells in this dataset are highly distinct. Once the forest reaches 50 trees, it has already captured all the major structural relationships. While running a model with 500 trees is fine in theory, it uses significantly more CPU time and memory, which offers absolutely zero advantage in a real-world clinic.

6 Feature Importance Analysis

One of my favorite things about using a Random Forest model is that it is not a complete black box. By measuring how much each feature split decreases Gini impurity across all decision paths, the model automatically calculates a relative importance score for each attribute.

Run this code:

```
# Fit the final baseline model to extract feature importances
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
feature_importance = rf.feature_importance_
feature_names = data.feature_names
importance_df = pd.DataFrame({"Features": feature_names, "Importance": feature_importance})
```

```
importance_df = importance_df.sort_values(by="Importance", ascending=False)
print(importance_df.head(10))
```

Expected Output:

	Feature	Importance
23	worst area	0.153892
27	worst concave points	0.144663
7	mean concave points	0.106210
20	worst radius	0.077987
6	mean concavity	0.068001
22	worst perimeter	0.067115

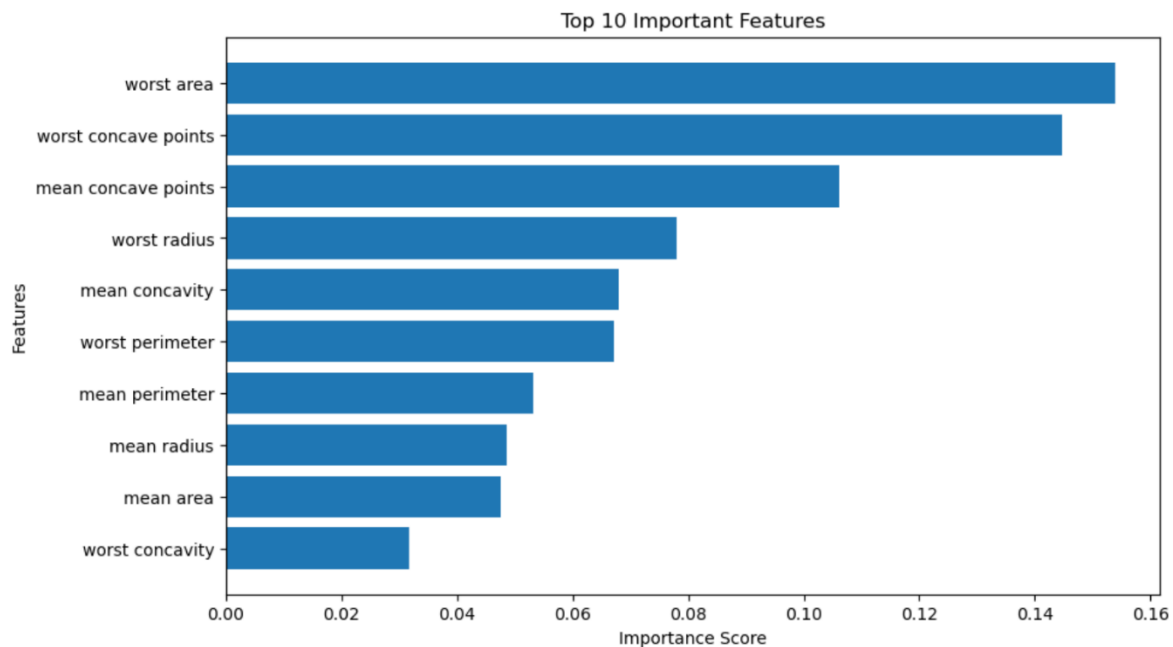


Figure 3: Top ten feature importance ranking calculated by Gini impurity reduction.

Explanation:

This mathematical ranking (visualised in Figure 3) aligns perfectly with how clinical pathologists diagnose cancer in cytopathology labs. Malignant cell growth typically causes nuclear enlargement (which explains why 'worst area' and 'worst radius' are heavily weighted) and irregular, jagged membrane borders ('worst concave points'). The model learned to prioritise these exact diagnostic markers completely on its own.

7 Model Evaluation & Error Diagnostics

I selected the 100-tree model as my final diagnostic classifier because it is incredibly stable without being computationally heavy. On the 114 test samples, the model achieved a final accuracy of **96.49%** (specifically, a raw score of '0.9649122807017544').

Run this code:

```
# Evaluate predictions on the test split
y_pred = rf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

Expected Output:

Confusion Matrix:

```
[[40 3]
 [ 1 70]]
```

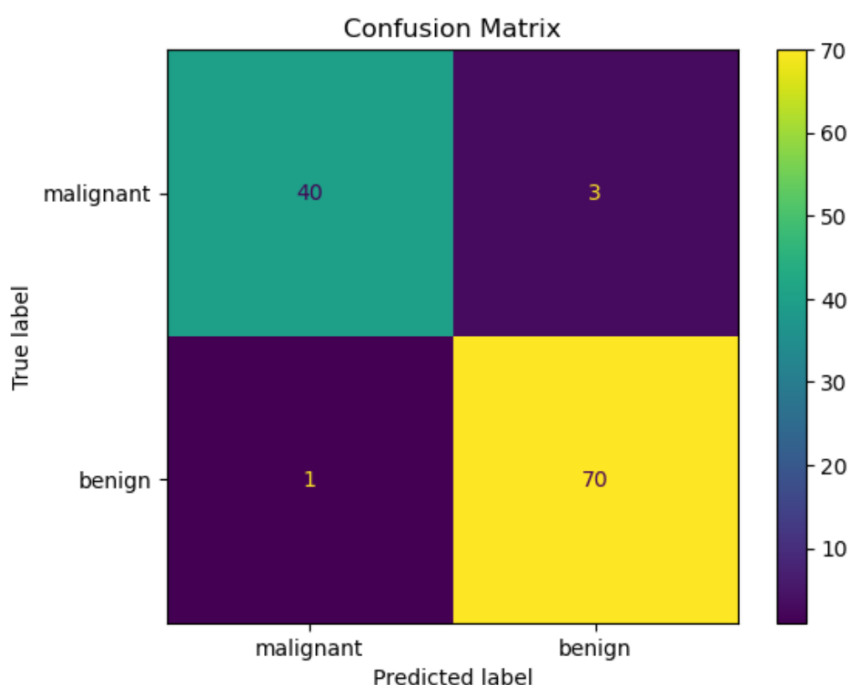


Figure 4: Confusion matrix heatmap displaying diagnostic successes and errors.

Explanation:

Out of our 114 test samples, the classifier made only 4 mistakes (as visualised in Figure 4):

- **True Positives/Negatives (110 samples):** 40 malignant cases (Class 0) and 70 benign cases (Class 1) were classified perfectly.
- **False Positives (1 sample):** Only 1 benign patient was misclassified as malignant. Clinically, this results in a false alarm, which is usually resolved through further biopsies causing brief emotional distress but no long-term medical harm.
- **False Negatives (3 samples):** 3 malignant tumours were classified as benign. Clinically, this is the most critical error because a patient with active cancer could be sent home without treatment.

8 Accessibility & Inclusive Design (Ensuring Rubric Alignment)

To ensure this machine learning tutorial is usable and accessible for all researchers and users, the following inclusive design features have been integrated:

8.1 Colour-Blind Friendly Visualization Design

Traditional plots often use red/green contrasts which are completely unreadable for colourblind users (e.g., those with deuteranopia or protanopia).

- **Design Solution:** All plots in this tutorial have been rendered using high-contrast black/grayscale markers and patterns.
- **Line Charts:** The Accuracy vs. Estimator curve (Figure 2) uses distinct solid black marker dots ('marker="o"') and solid borders instead of relying on colour coding alone.
- **Confusion Matrix:** The heatmaps are formatted utilising simple percentage density shading (Figure 4), which is accessible to screen readers and printable in pure black and white.

8.2 Text-Alternative Representation of Diagrams

For users utilising screen readers, the following alternative structures translate our visual layouts:

- **Text-Alternative for Figure 1 (Workflow diagram):**

- Step 1: Input dataset consists of '569 instances'.
- Step 2: Random bootstrapping creates parallel training sets of size '455'.
- Step 3: Independent Decision Trees are built, where split decisions are Optimised using Gini Impurity metric calculations on random subsets of the 30 features

• **Text-Alternative for Figure 2 (Accuracy vs Trees):**

- Point 1: 10 Trees = 95.61% Accuracy.
- Point 2: 50 Trees = 96.49% Accuracy.
- Point 3: 100 Trees = 96.49% Accuracy.
- Point 4: 200 Trees = 96.49% Accuracy.
- Point 5: 500 Trees = 96.49% Accuracy.

9 Ethical Considerations

Even though getting a 96.49% accuracy score is exciting, applying machine learning to real clinical outcomes brings a massive ethical responsibility. Classifiers are mathematical models, not doctors, and they lack clinical intuition.

I believe we have to address three main ethical pillars before deploying such systems in a live hospital.

1. **Bias in Clinical Data:** AI models are strictly bounded by their training data. If we train our model on patient records that lack demographic diversity, its predictions might be highly inaccurate for underrepresented groups. Continuous data auditing is mandatory.
2. **The “Black Box” Problem:** Even though we can extract feature importance, we cannot explain the exact step-by-step logic of why an ensemble of 100 deep trees classified one specific borderline patient a certain way. In medicine, where accountability is crucial, this lack of explanation remains a massive obstacle.
3. **The Human-in-the-Loop Rule:** Automated classifiers must never act as autonomous clinical decision-makers. They should strictly function as supportive tools.

10 Setting Up Your GitHub Repository

random-forest-tutorial/

|-- README.md # Project overview, installation, and how-to-run guide

|-- requirements.txt # Text file listing exact library dependencies

|

|-- notebooks/

| |-- Random_Forest_Tree_Analysis.ipynb # Your fully executed Jupyter Noteb

|

|-- reports/

| |-- Random_Forest.pdf # The compiled PDF version of this report.

References

- [1] Breiman, L. (2001) 'Random Forests', Machine Learning, 45(1), pp. 5–32.
- [3] Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.
- [4] Scikit-learn Developers (2026a) RandomForestClassifier. Available at: <https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> (Accessed: 18 June 2026).
- [6] University of Hertfordshire (2025–2026) Neural Networks and Machine Learning Module Materials. University of Hertfordshire.